

Introduction:

Unified Modeling Language (UML) is a standardized visual language used in object-oriented analysis and design to represent the structure and behavior of a system. IT helps developers and stakeholders understand how system components interact and how data flows through the system.

UML Diagram:

Unified Modeling Language (UML) is a standardized modeling language used to visualize, specify, construct, and document the components of a system. A UML diagram is a graphical representation that illustrates the structure and behavior of a system, including its main actors, roles, actions, and classes. UML provides a modern and comprehensive approach to software modeling and documentation. These diagrams help in understanding, designing, maintaining, and communicating the architecture of a system. UML diagrams serve as a blueprint for system development, requiring only the actual coding for implementation. There are various types of UML diagrams, each serving a specific purpose. These are broadly classified into two main categories:

1. **Structural Diagrams:** It describes the static aspects of the system (e.g., class diagrams, component diagram)
2. **Behavioral Diagrams:** These capture the dynamic behavior of the system (e.g., use case diagram, sequence diagram)

Use Case Diagram:

Use case diagrams were initially created for object-oriented UML but are now applied to various system development techniques. A use case model, which represents a system logically, shows what a system does without delving into how it does it. This model presents the system from the perspective of an external user, providing an abstract view of the system's functionality. It can be applied to both System Development Life Cycle (SDLC) and agile modeling.

In a use case model, the activities, services, and responses (use cases) that are significant to the system's users are highlighted separately from the system's internal functioning. Use cases

illustrate high-level functionality and the ways in which a user would interact with the system. They form the cornerstone of Unified Modeling Language (UML) modeling.

The following symbols are frequently used in use case diagrams:

- **Use case:** In system analysis, a use-case technique is used to clarify, identify, and organize system requirements.



Figure 1: Use Case

- **Actor:** In the context of UML use case diagrams, an actor represents any external entity that interacts with the system. Actors are not part of the system itself but engage with it to achieve certain goals or perform specific roles. They can be users, external systems, hardware devices, or other entities that require interactions with the system to fulfill a use case.

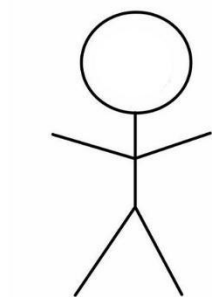


Figure 2: Actor

Here is the use case diagram of our system.

1. JIRA:

Actors:

- **Project Manager:** Manages overall project tasks, plans sprints, creates projects and tasks, assigns tasks, generates reports, and tracks progress.
- **Client:** Can log into the system to access relevant information or features.
- **Developer:** Fixes bugs, updates task status, tracks progress, updates status, and participates after login.
- **Tester:** Reports bugs, updates status, tracks progress, and participates after login.

Use Cases:

- **Login (Client, Project Manager, Developer, Tester):** Authentication process for all users to access the system.
- **Create Project (Project Manager):** Initiate and set up new projects.
- **Plan Sprint (Project Manager):** Organize and schedule sprint cycles.
- **Create Task (Project Manager):** Define tasks to be completed.
- **Assign Task (Project Manager):** Allocate tasks to developers or team members.
- **Generate Report (Project Manager):** Produce reports on project status and progress.
- **Track Progress (Project Manager, Developer, Tester, Client):** Monitor task and project progress.
- **Fix Bugs (Developer):** Resolve issues identified in the project.
- **Log Time (Developer):** Record time spent working on tasks.
- **Update Status (Developer, Tester):** Change the status of tasks or bugs.
- **Report Bug (Tester):** Submit bug reports for identified issues.

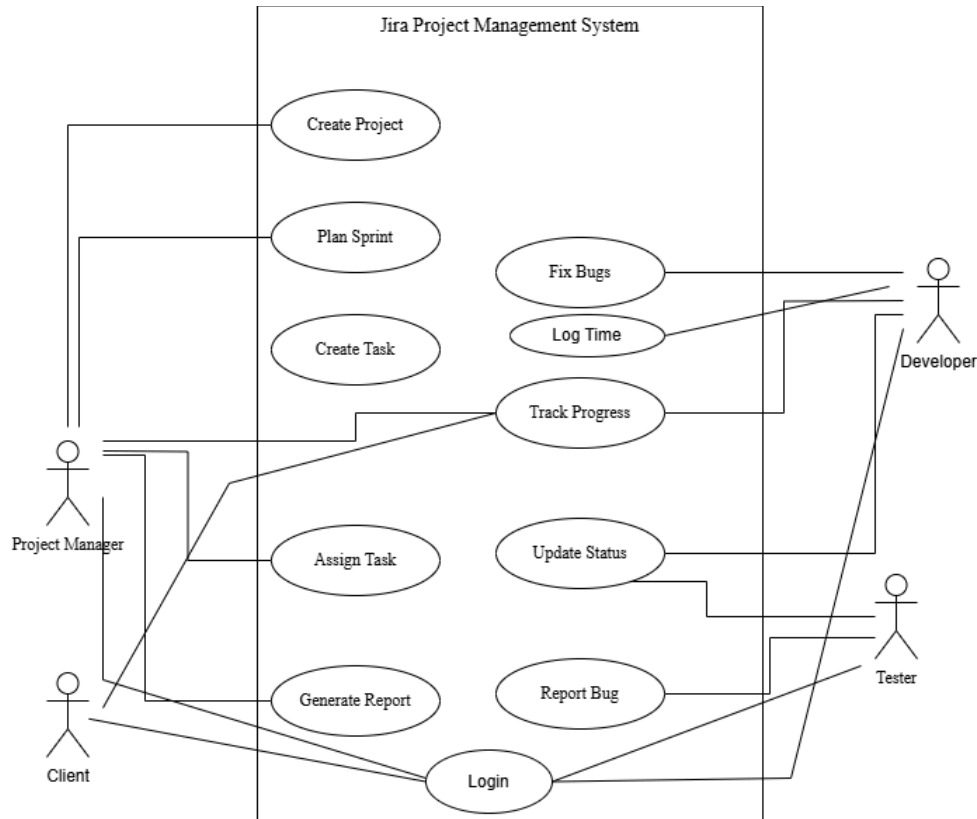


Figure 3: Use case diagram of JIRA

2. ML Staffing System:

Actors:

- **HR Manager:** Evaluates skill requirements, generates hire and reduction recommendations, considers budget constraints, provides reasoning, generates staffing reports, and assesses team performance.
- **Project Manager:** Considers budget constraints, provides reasoning, generates staffing reports, and assesses team performance.
- **ML Model:** Analyzes current staffing, predicts future needs, generates recommendations, performs model training and updates, calculates confidence scores, and conducts risk assessment analysis.

Use Cases:

- **Evaluate Skill Requirement (HR Manager):** Assesses the skills needed for staffing.

- **Generate Hire Recommendations (HR Manager, ML Model):** Suggests potential hires based on analysis; includes Analyze Current Staffing.
- **Analyze Current Staffing (ML Model):** Evaluates existing staffing levels; included in Generate Hire Recommendations and Generate Reduction Recommendations.
- **Generate Reduction Recommendations (HR Manager, ML Model):** Suggests staff reductions if needed; includes Predict Future Needs.
- **Predict Future Needs (ML Model):** Forecasts staffing requirements.
- **Consider Budget Constraints (HR Manager, Project Manager):** Evaluates financial limits impacting staffing decisions.
- **Provide Reasoning (HR Manager, Project Manager):** Offers explanations for staffing recommendations.
- **Generate Staffing Reports (HR Manager, Project Manager):** Produces reports on staffing status and decisions.
- **Assess Team Performance (HR Manager, Project Manager):** Reviews team effectiveness.
- **Model Training and Updates (ML Model):** Trains and updates the machine learning model.
- **Calculate Confidence Scores (ML Model):** Computes confidence levels for predictions.
- **Risk Assessment Analysis (ML Model):** Evaluates risks related to staffing decisions.

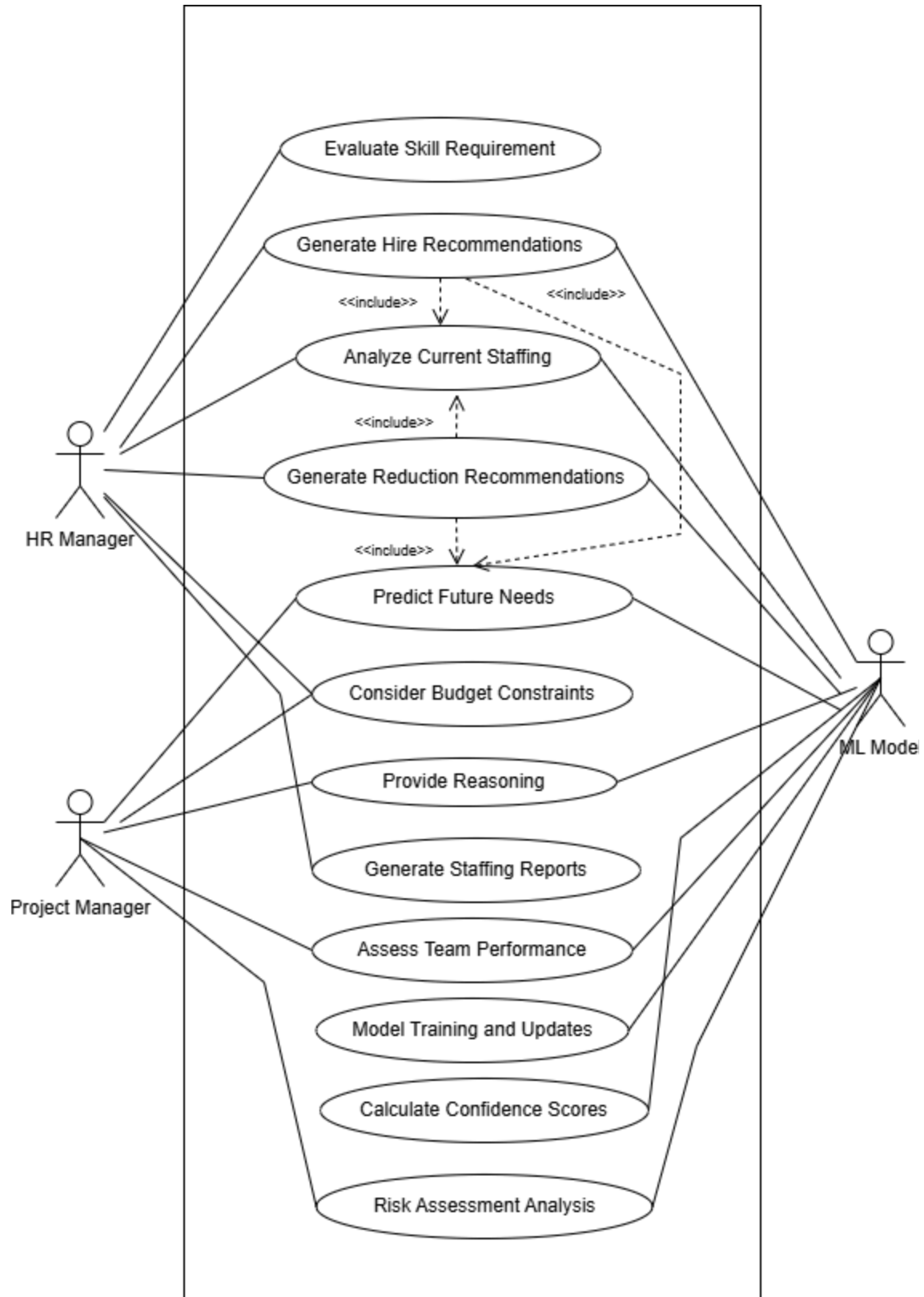


Figure 4: ML based Staffing System

3. Workload:

Actors:

- **System Admin:** Sets workload thresholds and manages system settings related to workload management.
- **Employee:** Views workload status and tracks their working hours.
- **Project Manager:** Monitors capacity, tracks workloads, generates workload reports, and views status.
- **Team Lead:** View status, and track hours worked.

Use cases:

- **Set Thresholds (System Admin):** Defines workload limits to manage employee capacity and prevent overload.
- **Monitor Capacity (Project Manager):** Oversees current workload capacity and utilization.
- **Track Workload (Project Manager, Team Lead):** Monitors the workload status and progress. Includes **Generate Reports** and **Monitor Capacity** as included use cases.
- **Generate Reports (Project Manager):** Produces reports related to workload distribution and resource utilization.
- **View Status (Employee, Project Manager, Team Lead):** Allows viewing the current workload and progress status.
- **Track Hours (Employee, Team Lead):** Records the number of hours worked by employees.

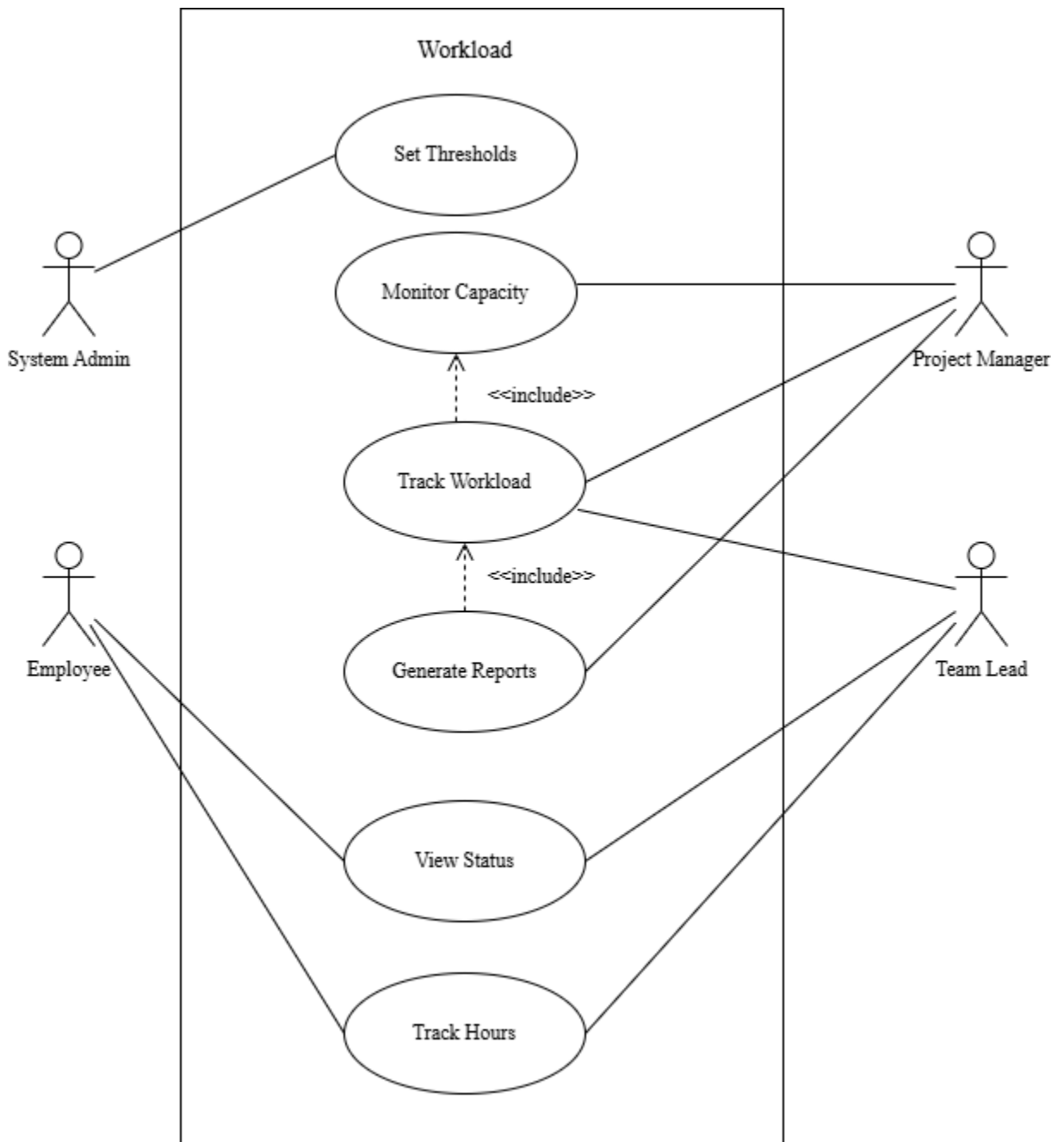


Figure 5: Workload Management

4. External Factor Integration:

Actors:

- **HR Manager:** Tracks vacation trends to understand employee leave patterns.

- **System Admin:** Configures data sources, imports holiday schedules, and processes weather data.
- **Project Manager:** Monitors the impact of the pandemic on projects and resources.
- **CEO/CTO:** Analyzes market conditions to guide strategic decisions.
- **External APIs:** Provide external data such as holiday schedules and weather information.

Use Cases:

- **Track Vacation Trends (HR Manager):** Monitors employee vacation patterns to optimize resource planning.
- **Import Holiday Schedule (System Admin):** Imports official holiday data and includes configuration of data sources from external APIs.
- **Configure Data Sources (System Admin):** Sets up and manages connections to external data providers.
- **Process Weather Data (System Admin):** Processes weather-related data sourced from external APIs.
- **Monitor Pandemic Impact (Project Manager):** Tracks how the pandemic affects project progress and workforce availability.
- **Analyze Market Conditions (CEO/CTO):** Reviews market trends and conditions to support business strategy.



Figure 6: External Factor Integration

5. System Integration:

Actors:

- **System Admin:** Responsible for handling API errors and managing connection to the Jira API.
- **System Developer:** Manages data consistency and ensures smooth synchronization of task data.
- **Jira System:** External system integrated with, providing task data synchronization.

Use Cases:

- **Handle API Errors (System Admin):** Manages and resolves errors arising from API communication issues.
- **Connect to Jira API (System Admin):** Establishes and maintains connection with the Jira system API. This use case includes handling API errors.
- **Sync Task Data (Jira System):** Synchronizes task-related data between systems.
- **Manage Data Consistency (System Developer):** Ensures that synchronized data remains consistent and accurate. This use case is included with Sync Task Data.

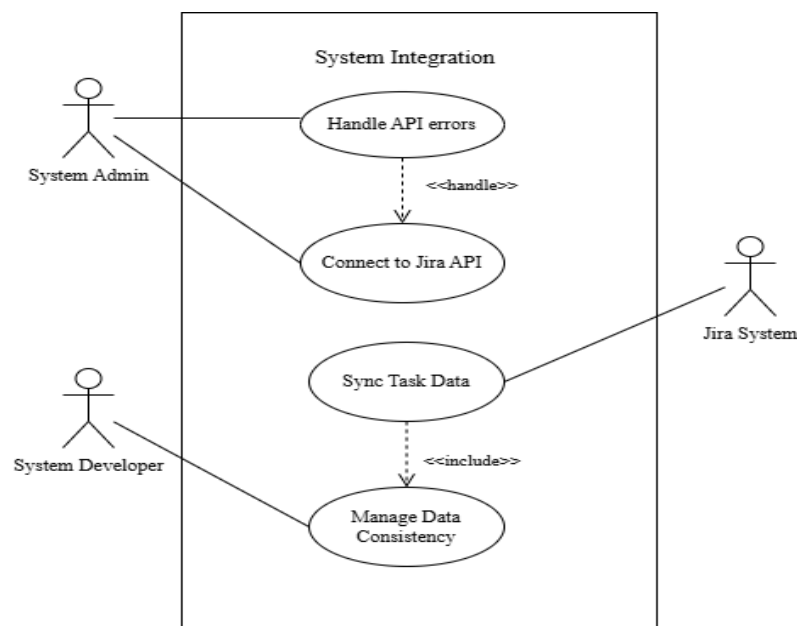


Figure 7: System Integration

6. Dashboard:

Actors:

- **HR Manager:** Shares reports, creates reports, exports data, displays metrics, visualizes trends, and manages access.
- **CEO/CTO:** Shares reports, creates reports, exports data, displays metrics, and visualizes trends.
- **Project Manager:** Shares reports, creates reports, exports data, displays metrics, and visualizes trends.
- **System Admin:** Manages access rights for the dashboard system.
- **Team Lead:** Involved in creating reports and displaying metrics.
- **Employee:** Views displayed metrics on the dashboard.

Use Cases:

- **Manage Access (System Admin):** Controls user permissions and access to the dashboard features.
- **Share Reports (HR Manager, CEO/CTO, Project Manager):** Distributes generated reports to relevant stakeholders.
- **Create Reports (HR Manager, CEO/CTO, Project Manager, Team Lead):** Generates reports based on collected data; includes **Export Data** and **Display Metrics** use cases.
- **Export Data (HR Manager, CEO/CTO, Project Manager):** Exports report or dashboard data for external use; included in Create Reports and connected with Display Metrics.
- **Display Metrics (HR Manager, CEO/CTO, Project Manager, Team Lead, Employee):** Shows key performance indicators and metrics on the dashboard.
- **Visualize Trends (HR Manager, CEO/CTO, Project Manager):** Provides graphical trend analysis of data.

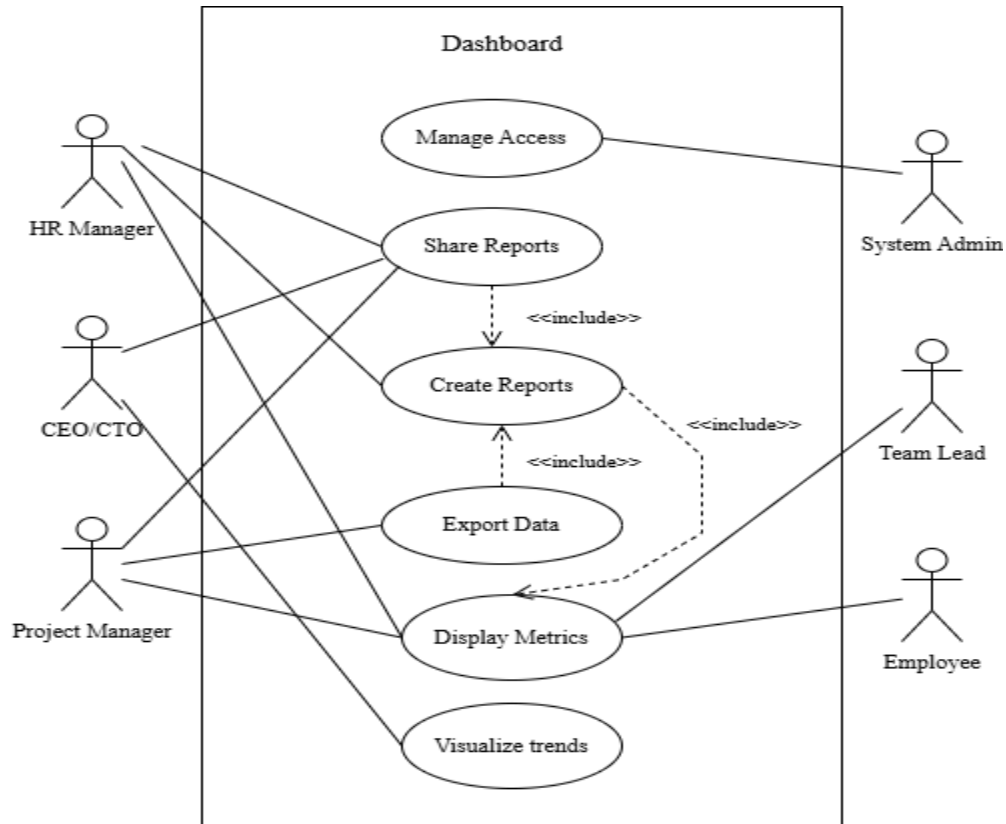


Figure 8: Dashboard of ML System

7. Notification Alert:

Actors:

- **Project Manager:** Creates announcements, sends alerts, and tracks notifications.
- **HR Manager:** Sends alerts and tracks notifications.
- **Team Lead:** Sends alerts and tracks notifications.
- **Employee:** Tracks notifications and manages personal preferences.
- **System Admin:** Sends emails, manages preferences, and tracks notifications.

Use Cases:

- **Track Notifications (Project Manager, HR Manager, Team Lead, Employee, System Admin):** Monitors system notifications and alerts.

- **Create Announcement (Project Manager):** Prepares announcements to be sent to users; includes sending emails.
- **Send Emails (Project Manager, System Admin):** Sends email communications; included in Create Announcement and Send Alerts.
- **Send Alerts (Project Manager, HR Manager, Team Lead, System Admin):** Sends alert notifications to users; includes Send Emails.
- **Manage Preference (Employee, System Admin):** Allows managing notification preferences and settings.

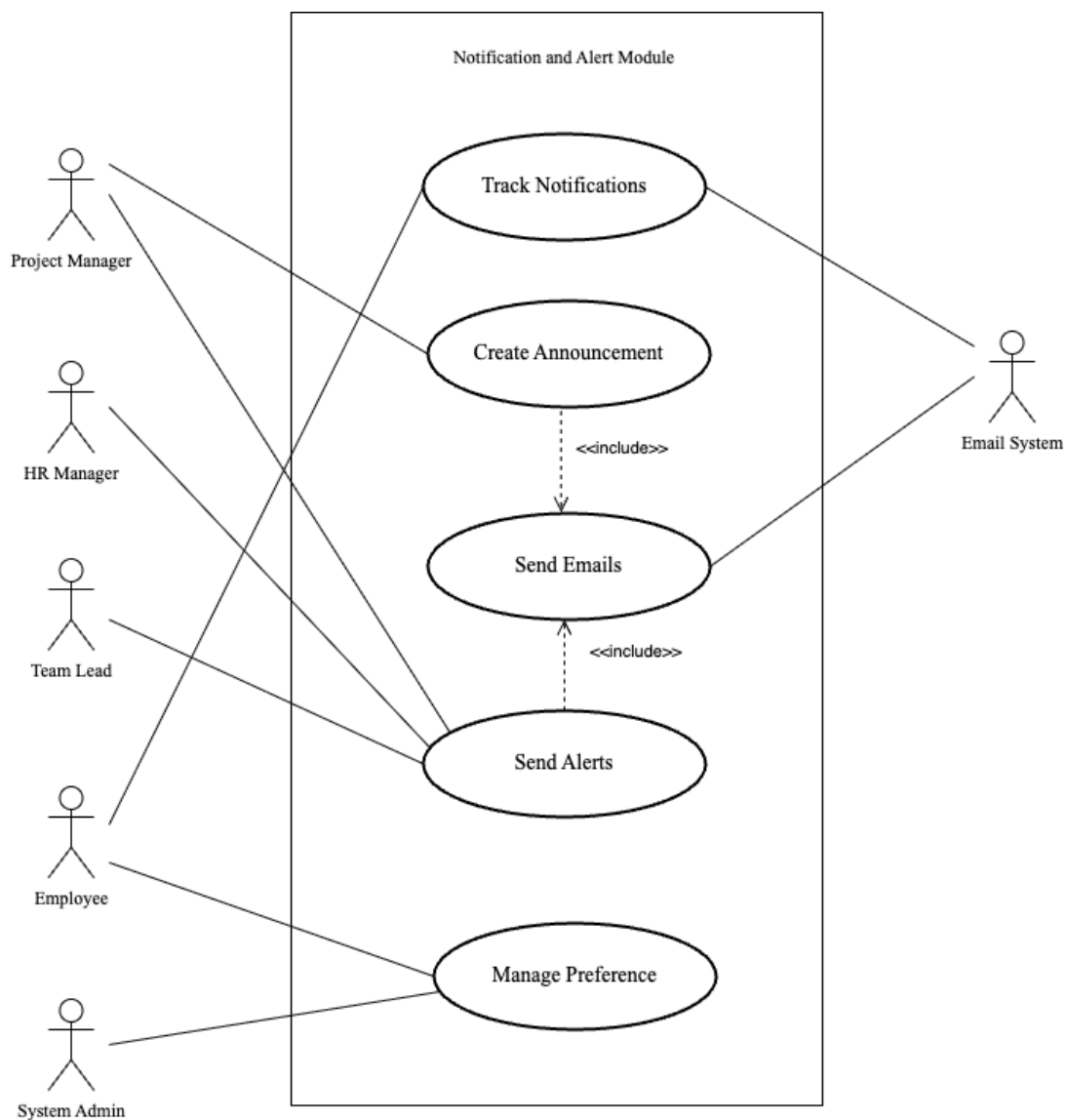


Figure 9: Notification Alert of ML System

Conclusion:

This report has focused on the use case diagram for the proposed ML-based staffing recommendation system at TORK. The diagram identifies the key actors—such as the project manager, HR manager, and external systems like Jira—and outlines their interactions with the system.

By visually representing the system's primary functionalities, the use case diagram helps clarify the system's goals and user expectations. It serves as an essential step in understanding the scope of the system and sets the foundation for further design and development in the upcoming phases.